

DeepHedger-Default: Guided Recipe Search for Deep Hedging under Market Frictions

Anonymous Author(s)

Abstract

Deep hedging replaces a hand-designed hedge with a policy trained end to end against a risk functional, but reported performance is sensitive to representation, initialization, state design, and optimization. We study a different question from proposing another isolated architecture: can a small, reproducible sequence of validated choices turn the standard pathwise deep-hedging policy into a strong default? Our protocol is inspired by recipe-oriented tabular learning. It adds one component family at a time—periodic numerical embeddings, residual prediction around a model hedge, running profit and loss, modern optimizers, and stochastic exploration—and freezes each choice using a development panel that is disjoint from the final benchmark. The selected policy is then compared with information-matched analytical and numerical hedges on one-factor-at-a-time slices of GBM, Heston, and rough-Bergomi markets, two payoffs, transaction costs, moneyness, and rebalancing frequencies. The main comparator under Heston is the local minimum-variance delta, not merely the price delta. We specify paired multi-seed evaluation, dimensionless risk normalization, tail-risk and turnover metrics, and a one-class software interface. Preliminary single-execution Heston results motivate the study but are not treated as confirmatory evidence; the paper’s claims are intentionally conditional on the locked multi-seed benchmark.

CCS Concepts

• **Computing methodologies** → **Reinforcement learning**; *Neural networks*; • **Applied computing** → *Economics*.

Keywords

deep hedging, transaction costs, stochastic volatility, numerical embeddings, minimum-variance delta, reproducible benchmarking

ACM Reference Format:

Anonymous Author(s). 2026. DeepHedger-Default: Guided Recipe Search for Deep Hedging under Market Frictions. In *Proceedings of ACM International Conference on AI in Finance (ICAIF '26)*. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Deep Hedging trains a parametric trading policy directly against a portfolio risk measure and can incorporate transaction costs, liquidity restrictions, and other non-classical frictions [4]. This flexibility makes it an attractive alternative to deriving a new control solution for every market and contract. It also creates a practical ambiguity: two implementations of “Deep Hedging” can differ in input scaling, state variables, output parameterization, initialization, optimizer, and exploration, with each choice materially changing convergence.

The usual response is to introduce one more learning algorithm. We instead treat the complete training stack as the object of study. This follows the recipe-oriented logic of RealMLP, where carefully

validated defaults and broad evaluation made a simple model competitive without per-dataset tuning [12]. In hedging, the analogous deliverable is a policy that an end user can instantiate once, while market dynamics, payoff, and risk preferences remain explicit configuration.

This framing raises a stronger baseline requirement. The plain Heston price delta, $\partial C / \partial S$, ignores the locally hedgeable component of variance shocks when spot and variance are correlated. A stock-only minimum-variance (MV) hedge corrects this through variance sensitivity [13]. Therefore, beating a Black–Scholes or price-delta baseline is not sufficient evidence for a strong recipe. The central empirical question is whether the recipe matches or improves upon the best information-matched mathematical hedge reliably, and where it fails, rather than whether it wins in one favorable training run.

The intended contributions are:

- (1) a guided, stagewise search protocol that attributes gains to representation, anchor, state, optimizer, and stochasticity while keeping the final benchmark locked;
- (2) a fair baseline suite centered on the MV delta and cost-aware no-trade policies, with the same state information available to learned and mathematical hedges;
- (3) a multi-market, multi-contract benchmark using paired seeds and reporting risk, tail loss, turnover, fees, and deviation from the MV hedge; and
- (4) a reusable DeepHedger class whose selected recipe is exposed as a documented default rather than notebook-specific code.

The current manuscript is a mark-0 draft. It fixes the claims, experimental questions, table structure, and evidence standard before the confirmatory runs. Numerical values in Section 6 are preliminary diagnostics with explicit provenance.

2 Related Work

Deep and reinforcement-learning hedges. Buehler et al. formulate hedging with frictions as direct optimization of convex risk measures over neural policies [4]. Subsequent work studies continuous-control and distributional objectives [6], actor–critic formulations that cover multiple portfolios or risk aversions [5, 19], and policies trained on empirical intraday data [17]. These works establish that learning can outperform conventional deltas in suitable settings. They do not remove the need for controlled recipe selection against the strongest model-consistent hedge.

Structured policies and optimization. Residual learning around a Black–Scholes delta can reduce the complexity of the learned function [20]. K-FAC approximates neural curvature with tractable Kronecker factors [16]; recent work applies related second-order structure to deep hedging and reports faster optimization on a

difficult cliquet problem [18]. The current study asks whether residual anchors and curvature-aware optimization remain useful after representation and state design are controlled.

Recipe transfer from tabular learning. Numerical feature embeddings with periodic activations can improve simple MLPs [8]; Fourier features provide a related mechanism for representing high-frequency low-dimensional functions [21]. More recently, a benchmark of fifteen optimizers on seventeen tabular datasets identified Muon, Schedule-Free AdamW, and AdamW with an exponential moving average as strong candidates [9]; Schedule-Free learning itself removes the need to prescribe the stopping horizon in advance [7]. These results motivate candidates, but do not imply that their ranking transfers to a path-dependent entropic hedging objective. That transfer is an empirical question.

3 Hedging Problem

We work on a discrete grid $0 = t_0 < \dots < t_N = T$. Let $B_t = \exp(rt)$, $\bar{S}_t = B_t^{-1}S_t$, and $\bar{H} = B_T^{-1}H(S_T)$. Before the move from t_i to t_{i+1} , a policy chooses δ_i shares. In discounted units, with proportional cost c and $\delta_{-1} = 0$, terminal hedging profit and loss is

$$\bar{Z}^\delta = \sum_{i=0}^{N-1} \delta_i (\bar{S}_{t_{i+1}} - \bar{S}_{t_i}) - c \sum_{i=0}^{N-1} \bar{S}_{t_i} |\delta_i - \delta_{i-1}| - \bar{H}. \quad (1)$$

An initial option premium would shift every strategy by the same constant and does not affect their ranking under CARA/entropic risk, although it must be restored for economically interpretable wealth levels. Each contract declares an ex-ante currency scale s_H : underlying notional for a vanilla call and cash payout for a digital. We set $\tilde{Z} = \bar{Z}/s_H$ and use dimensionless risk aversion α . This rule is fixed before simulation, never estimated from test PnL, and prevents raw digital and vanilla notionals from silently changing risk preferences. The training objective is

$$\mathcal{L}_\alpha(\delta) = \text{SoftMin}_\alpha(\tilde{Z}^\delta) = \frac{1}{\alpha} \log \mathbb{E} \left[\exp(-\alpha \tilde{Z}^\delta) \right], \quad \alpha > 0, \quad (2)$$

where lower is better. We do not describe this quantity as CVaR: final evaluation reports lower-tail CVaR₉₅ and CVaR₉₉ separately.

CARA and running PnL. Entropic utility has constant absolute risk aversion: adding current wealth or a fixed premium translates the certainty equivalent but does not change the optimal action. Running PnL is therefore not justified as wealth-dependent risk preference. It can help only as a proxy for omitted path information, finite-sample optimization, or a state variable introduced by the precise accounting/control formulation. We triangulate the mechanism using independent retraining with the feature on/off, inference-time masking, and a fixed-state diagnostic that holds $(t_i, S_i, q_i, \delta_{i-1})$ constant while sweeping $\tilde{G}_i$ over empirical quantiles. A reproducible seeded gain may identify omitted history or an optimization scaffold, in which case the feature is retained and its mechanism characterized. Wealth-dependent behavior, however, requires explicit constraints or a non-CARA objective.

3.1 Information-matched mathematical hedges

In undiscounted units, the Heston dynamics are

$$dS_t = rS_t dt + S_t \sqrt{v_t} dW_t^S, \quad (3)$$

$$dv_t = \kappa(\theta - v_t) dt + \xi \sqrt{v_t} dW_t^v, \quad d\langle W^S, W^v \rangle_t = \rho dt, \quad (4)$$

let C denote the option price as a function of time, spot, and variance. We compute it from the characteristic-function representation [11]. Minimizing the instantaneous conditional variance of $dC - \delta dS$ gives

$$\delta_{MV} = C_S + \frac{\rho \xi}{S} C_v. \quad (5)$$

The implementation evaluates C , C_S , and C_v in batched Torch code using fixed Gauss–Legendre quadrature and automatic differentiation. Crucially, the network observes v_t whenever the baseline uses it. The comparison therefore does not give the learned policy an unobserved-volatility advantage.

Transaction costs can make continuous retargeting to Equation (5) suboptimal. We consequently include a validation-tuned no-trade band around the MV delta. “Best mathematical baseline” means the predeclared, validation-selected member of this baseline family; it never means choosing the best test-set row after evaluation. Under rough Bergomi, Keller-Ressel derives analytic expressions for the variance-optimal strategy and its mean-square error [2, 15]. We include rough Bergomi in the headline benchmark only after the simulator’s filtration and state representation are matched to that result and our implementation agrees with both the analytic expression and a converged conditional-covariance calculation. Otherwise those rows are deferred rather than compared against a weak proxy.

4 DeepHedger-Default

4.1 One policy class, explicit configuration

At time t_i , the deterministic policy consumes

$$x_i = [\log(S_{t_i}/K), q_i, \tau_i/T, \delta_{i-1}, \mathbf{1}_{\text{PnL}} \tilde{G}_i], \quad (6)$$

where q_i contains market state variables (for example v_i), τ_i is time to maturity, and $\tilde{G}_i$ is normalized running portfolio PnL. Log-moneyness replaces raw spot to make scale transfer an explicit inductive bias. The public class exposes each term in Equation (6), the architecture, anchor, optimizer, and stochastic trainer as hyperparameters; the paper ultimately selects one default configuration.

Two periodic representations are compared with a plain MLP. A matrix-mixed embedding uses

$$e_{\text{mix}}(x) = [\sin(xB), \cos(xB)],$$

while the feature-wise form assigns frequencies independently,

$$e_{\text{fw}}(x_j) = [\sin(2\pi x_j c_j), \cos(2\pi x_j c_j)].$$

Both frequency parameters are trainable. Initial scale σ , number of frequencies, and the inclusion of the linear projection are tuned inside the representation stage, rather than fixed from a different application domain.

The output can be direct or residual:

$$\delta_i = \delta_{\text{anchor}}(t_i, S_{t_i}, q_i) + f_\theta(x_i). \quad (7)$$

The candidate anchors are a local-GBM delta and the strongest information-matched model hedge. Zero initialization of the last

layer makes the initial policy exactly equal to the anchor. This separates learning a correction from rediscovering the full hedge.

4.2 Guided search, not a combinatorial sweep

Figure 1 replaces a full Cartesian grid with an auditable sequence of conditional questions. Each stage uses common random numbers across candidates and aggregates ranks across a fixed development panel. A candidate is promoted only if its paired mean improvement is positive and its uncertainty is acceptable; otherwise the incumbent is retained. The selection log records all candidates, so negative transfer remains visible.

Greedy search can miss interactions. We therefore reserve a small interaction audit after Stage 5: remove each selected component once, and test the two most plausible pairwise interactions (embedding–anchor and anchor–PnL). This retains the interpretability and compute advantages of guided search without pretending that component effects are additive.

Optimizer stage. The candidate set is Adam, AdamW, K-FAC, Schedule-Free AdamW, AdamW with weight EMA, and Muon [14]. The 2026 tabular benchmark is a motivation for including the last three, not evidence that they will win in hedging. Every optimizer receives a comparable learning-rate and regularization budget; wall-clock time and simulator paths are reported alongside epochs.

Stochastic stage. The current prototype adds Gaussian action noise and an annealed entropy bonus to the pathwise objective. This is a maximum-entropy pathwise policy, not full Soft Actor-Critic: SAC is specifically an off-policy actor–critic algorithm with a learned critic [10]. The final paper will use the label SAC only for an implementation with critic, targets, and replay. The stage tunes initial action standard deviation, entropy strength, and annealing; it compares the deterministic incumbent, the pathwise MaxEnt policy, and full SAC if the latter passes independent smoke tests.

5 Experimental Protocol

5.1 Development and locked benchmark

Recipe construction uses a predeclared, stratified Heston development panel covering low/high vol-of-vol, zero/nonzero cost, in/out-of-the-money calls, and short/long rebalancing grids. The panel is fixed before Stage 1. Five fully seeded runs per candidate are used during selection. Torch CPU/CUDA, NumPy, initialization, simulator streams, and stochastic actions all receive explicit seeds.

The frozen recipe is then evaluated with ten fresh seeds on the isolated benchmark slices below. This is deliberately not the Cartesian product of all levels. Within each market, the reference cell is an at-the-money call with 10-bp costs, $N = 60$, and $\alpha = 1$; one axis changes at a time while every other coordinate stays at its reference. The exact slices and a small, predeclared cross-factor stress set are frozen before final evaluation.

The market slice follows complete $\rightarrow$ Markov stochastic volatility $\rightarrow$ non-Markov rough volatility. GBM with zero costs is a mandatory sanity check: with finer rebalancing, the learned hedge should approach the Black–Scholes hedge [3]. Heston is the main controlled incomplete market. Rough Bergomi tests whether a recipe selected in a Markov setting transfers to history-dependent volatility. A Bates jump-diffusion [1] is a useful optional stress test, but

Table 1: Locked one-factor benchmark slices around a reference cell; these levels do not define a Cartesian product. Rough-Bergomi rows are gated on reference-hedge validation.

Axis	Reference	Isolated slice
Market	Heston, $\xi = 0.3$	GBM; Heston $\xi \in \{0.1, 0.3, 0.5\}$; rough Bergomi
Payoff	European call	call; digital call
K/S_0	1.0	$\{0.9, 1.0, 1.1\}$
Cost	10 bp	$\{0, 5, 10, 30\}$ bp
Steps N	60	$\{30, 60, 120, 300\}$
Risk α	1	prespecified dimensionless sensitivities
Seeds	10	paired evaluation paths within every seed

will not displace the three primary market families unless compute and page budget permit.

5.2 Baselines, metrics, and statistical reporting

Every applicable slice cell contains: no hedge; model price delta; local MV delta; validation-tuned MV no-trade band; a converged numerical conditional-MV hedge when no analytic counterpart exists; standard Deep Hedging with the base MLP; and the frozen DeepHedger-Default. All policies use the same simulated state information and trading dates.

The primary endpoint is test SoftMin $_{\alpha}$ of dimensionless PnL. Secondary endpoints are normalized PnL mean and standard deviation, lower-tail CVaR $_{95}$ and CVaR $_{99}$, mean fees, turnover $\sum_i |\delta_i - \delta_{i-1}|$, training wall time, and

$$D_{MV} = \frac{\mathbb{E} \sum_i S_{t_i} (\delta_i - \delta_{MV,i})^2}{\mathbb{E} \sum_i S_{t_i}}, \quad (8)$$

which measures economically weighted deviation from the local MV hedge. Results are mean $\pm$ standard deviation over seeds. The main comparison also reports the paired seed-level gap and a 95% bootstrap confidence interval. Tables retain every prespecified benchmark cell; aggregate ranks cannot hide a failed market or payoff.

6 Preliminary Evidence

Table 3 is included to test the result narrative and table design, not to establish the contribution. It comes from one July 2026 Heston notebook execution with $S_0 = K = 100$, $v_0 = \theta = 0.04$, $r = 0.01$, $\kappa = 2$, $\xi = 0.1$, $\rho = -0.7$, $T = 1$, $N = 30$, $a = 1$, 5,000 held-out paths, and 10,000 training epochs. Evaluation paths were initialized from NumPy seed 42, but Torch initialization and all stochastic sources were not fully seeded. There is therefore no valid seed-level uncertainty estimate.

The notebook simulates $r = 0.01$ but calls the backtest without cash-account accrual and does not first discount paths and payoff. These values therefore inherit an unresolved accounting inconsistency in addition to their seed limitations. They are diagnostics only and will be regenerated using the coherent convention in Equation (1).

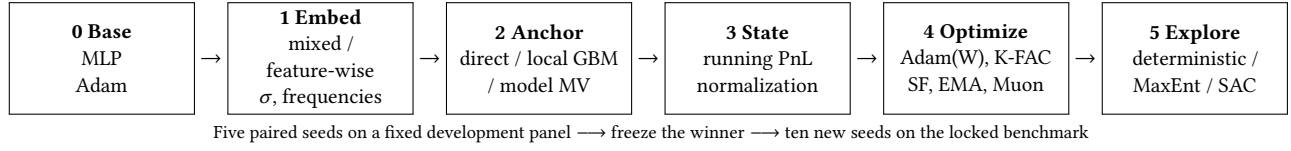


Figure 1: The guided-search ladder. At stage k , only the displayed component family changes; all decisions from stages $< k$ are frozen. The final benchmark is not queried during recipe construction.

Table 2: Planned component-isolating ablation. Each row changes one stage relative to its incumbent. This table will be populated from the development panel, not from the final benchmark.

Stage	Candidate	SoftMin	Paired gap	CVaR ₉₉	Turnover	Time	Decision
1	Mixed vs. feature-wise periodic; σ HPO	TBD	TBD	TBD	TBD	TBD	TBD
2	Direct vs. local-GBM vs. MV residual	TBD	TBD	TBD	TBD	TBD	TBD
3	Running PnL off/on	TBD	TBD	TBD	TBD	TBD	TBD
4	Adam(W), K-FAC, SF-AdamW, AdamW+EMA, Muon	TBD	TBD	TBD	TBD	TBD	TBD
5	Deterministic, MaxEnt pathwise, full SAC	TBD	TBD	TBD	TBD	TBD	TBD
audit	Leave-one-component-out and two interactions	TBD	TBD	TBD	TBD	TBD	TBD

Table 3: Preliminary single-execution Heston test SoftMin (lower is better). Values are diagnostics only; “periodic” is the existing matrix-mixed embedding, and the residual anchor is a local-GBM proxy rather than MV.

Method	0 bp	10 bp
Heston price delta	9.2715	9.5526
Heston MV delta	9.1966	9.4879
DH, plain MLP	10.0053	10.1567
DH, periodic	9.0135	9.3076
DH, periodic residual	8.9572	9.1595
+ K-FAC	8.9613	9.2549
+ Muon	9.0665	9.3094
MaxEnt pathwise policy	8.9815	9.2014

Three observations shape the confirmatory study. First, the plain MLP is not a strong baseline in this cell. Second, periodic representation and residual prediction appear more consequential than changing the optimizer. Third, K-FAC is not uniformly best in the latest provenance-clear run, contrary to an earlier notebook result. This reversal is precisely why a single-seed “best-method” story is not defensible.

The MV comparison is not explained by a coarse characteristic-function integration in this cell. In a separate fixed-path budget sweep, 256 Gauss–Legendre nodes at truncation 200 had hedge-ratio RMSE 2.34×10^{-7} and maximum absolute deviation 7.92×10^{-5} relative to an 8,192-node, truncation-400 reference; SoftMin was stable at the reported precision. This check must be repeated over the final state grid and market parameters.

7 Reproducibility and Software Contract

Every run will be generated from a versioned configuration containing market, payoff, method, and seed. A per-run entry point writes a machine-readable report, status, and completion marker; aggregation reads only reports and produces paper tables without

notebook copy-and-paste. Repeating a completed configuration is idempotent. The release will include:

- deterministic seed handling for NumPy, Torch, CUDA, simulator streams, weight initialization, and stochastic actions;
- unit tests comparing batched Heston price/MV deltas with the retained SciPy reference on a grid of spot, variance, and maturity;
- an accounting equivalence test showing that discounted-path PnL and an undiscounted self-financing cash account produce the same result;
- configuration manifests for the development ladder and locked benchmark, plus the exact aggregation script; and
- a pip-installable DeepHedger whose constructor accepts the same explicit hyperparameters used by the experiments and whose recipe="default" resolves to the frozen selection.

The default should be a named, immutable recipe version. Adding every experimental branch to one class is valid only if configuration validation, serialization, and backward compatibility are tested; otherwise a nominally simple API merely hides irreproducible notebook state.

8 Limitations and Scope

The study trains a policy for a specified simulated market and contract. It does not claim a single amortized hedge for arbitrary portfolios, robustness to market misspecification, or real-data superiority. Those are separate questions. The greedy search is compute-efficient but can miss higher-order component interactions; the interaction audit only partially addresses this. Analytical and nested-Monte-Carlo baselines also encode model knowledge that may not be available in deployment, but this is deliberate: a method should first be judged against the strongest fair oracle in simulation before weaker-information comparisons are used.

Simulation access can make end-to-end pathwise gradients more sample-efficient than model-free RL, so SAC is not expected to win

Table 4: Headline benchmark table reserved for the frozen recipe. Each entry will be mean $\pm$ standard deviation over ten fresh seeds; the gap is paired DeepHedger-Default minus the validation-selected mathematical baseline.

Market	Payoff	Best mathematical	Base DH	DeepHedger-Default	Paired gap	95% CI	CVaR ₉₉ gap
GBM	call	TBD	TBD	TBD	TBD	TBD	TBD
GBM	digital	TBD	TBD	TBD	TBD	TBD	TBD
Heston	call	TBD	TBD	TBD	TBD	TBD	TBD
Heston	digital	TBD	TBD	TBD	TBD	TBD	TBD
rough Bergomi	call	TBD	TBD	TBD	TBD	TBD	TBD
rough Bergomi	digital	TBD	TBD	TBD	TBD	TBD	TBD

by default. A negative SAC result remains informative if implementation and tuning are credible. Finally, recent empirical work shows that learned delta corrections can be regime fragile [22]; interpretability metrics and broad simulated coverage reduce, but do not eliminate, that deployment risk.

9 Conclusion

This paper is designed around a claim that can fail cleanly. A useful DeepHedger-Default must survive seeded selection, match or improve aggregate performance against a strong information-matched hedge on a locked and heterogeneous benchmark, characterize the cells in which it fails, and expose the chosen recipe as reproducible software. It need not win every cell. The current evidence is sufficient to motivate the guided search, but not to declare a winner. *[Result pending: replace this paragraph with the confirmed cross-market finding and its uncertainty after the locked benchmark.]*

Generative AI Use Disclosure

OpenAI Codex was used to help organize and draft the mark-0 manuscript and to check the consistency of citations and repository evidence. The human author is fully responsible for the scientific claims, experiments, citation verification, and final text. This non-identifying section must remain visible in an anonymous review build unless the ICAIF 2026 policy explicitly prescribes another disclosure location or wording.

References

- [1] David S. Bates. 1996. Jumps and Stochastic Volatility: Exchange Rate Processes Implicit in Deutsche Mark Options. *The Review of Financial Studies* 9, 1 (1996), 69–107. doi:10.1093/rfs/9.1.69
- [2] Christian Bayer, Peter K. Friz, and Jim Gatheral. 2016. Pricing under Rough Volatility. *Quantitative Finance* 16, 6 (2016), 887–904. doi:10.1080/14697688.2015.1099717
- [3] Fischer Black and Myron Scholes. 1973. The Pricing of Options and Corporate Liabilities. *Journal of Political Economy* 81, 3 (1973), 637–654. doi:10.1086/260062
- [4] Hans Buehler, Lukas Gonon, Josef Teichmann, and Ben Wood. 2019. Deep Hedging. *Quantitative Finance* 19, 8 (2019), 1271–1291. doi:10.1080/14697688.2019.1571683
- [5] Hans Buehler, Phillip Murray, and Ben Wood. 2024. Deep Bellman Hedging. arXiv:2207.00932, version 4. doi:10.48550/arXiv.2207.00932
- [6] Jay Cao, Jacky Chen, John Hull, and Zissis Poulos. 2021. Deep Hedging of Derivatives Using Reinforcement Learning. *The Journal of Financial Data Science* 3, 1 (2021), 10–27. doi:10.3905/jfds.2020.1.052
- [7] Aaron Defazio, Xingyu Yang, Harsh Mehta, Konstantin Mishchenko, Ahmed Khaled, and Ashok Cutkosky. 2024. The Road Less Scheduled. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., Red Hook, NY, USA, 9974–10007. doi:10.52202/079017-0320
- [8] Yuri Gorishniy, Ivan Rubachev, and Artem Babenko. 2022. On Embeddings for Numerical Features in Tabular Deep Learning. In *Advances in Neural Information Processing Systems*, Vol. 35. Curran Associates, Inc., Red Hook, NY, USA, 24991–25004. https://proceedings.neurips.cc/paper_files/paper/2022/hash/9e9f0ffc3d836836ca96cbf8fe14b105-Abstract-Conference.html
- [9] Yuri Gorishniy, Ivan Rubachev, Dmitrii Feoktistov, and Artem Babenko. 2026. Benchmarking Optimizers for MLPs in Tabular Deep Learning. arXiv:2604.15297. doi:10.48550/arXiv.2604.15297
- [10] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *Proceedings of the 35th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 80)*. PMLR, Stockholm, Sweden, 1861–1870. <https://proceedings.mlr.press/v80/haarnoja18b.html>
- [11] Steven L. Heston. 1993. A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options. *The Review of Financial Studies* 6, 2 (1993), 327–343. doi:10.1093/rfs/6.2.327
- [12] David Holzmüller, Léo Grinsztajn, and Ingo Steinwart. 2024. Better by Default: Strong Pre-Tuned MLPs and Boosted Trees on Tabular Data. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., Red Hook, NY, USA, 26577–26658. doi:10.52202/079017-0837
- [13] John C. Hull and Alan White. 2017. Optimal Delta Hedging for Options. *Journal of Banking & Finance* 82 (2017), 180–190. doi:10.1016/j.jbankfin.2017.05.006
- [14] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesima, Laker Newhouse, and Jeremy Bernstein. 2024. Muon: An Optimizer for Hidden Layers in Neural Networks. <https://kellerjordan.github.io/posts/muon/>
- [15] Martin Keller-Ressel. 2022. Bartlett’s Delta Revisited: Variance-Optimal Hedging in the Lognormal SABR and in the Rough Bergomi Model. arXiv:2207.13573. doi:10.48550/arXiv.2207.13573
- [16] James Martens and Roger Grosse. 2015. Optimizing Neural Networks with Kronecker-factored Approximate Curvature. In *Proceedings of the 32nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 37)*. PMLR, Lille, France, 2408–2417. <https://proceedings.mlr.press/v37/martens15.html>
- [17] Oskari Mikkilä and Juho Kanninen. 2023. Empirical Deep Hedging. *Quantitative Finance* 23, 1 (2023), 111–122. doi:10.1080/14697688.2022.2136037
- [18] Konrad Mueller, Amira Akkari, Lukas Gonon, and Ben Wood. 2024. Fast Deep Hedging with Second-Order Optimization. In *Proceedings of the 5th ACM International Conference on AI in Finance*. Association for Computing Machinery, New York, NY, USA, 319–327. doi:10.1145/3677052.3698604
- [19] Phillip Murray, Ben Wood, Hans Buehler, Magnus Wiese, and Mikko S. Pakkanen. 2022. Deep Hedging: Continuous Reinforcement Learning for Hedging of General Portfolios across Multiple Risk Aversions. In *Proceedings of the 3rd ACM International Conference on AI in Finance*. Association for Computing Machinery, New York, NY, USA, 361–368. doi:10.1145/3533271.3561731
- [20] Chunhui Qiao and Xiangwei Wan. 2024. Enhancing Black–Scholes Delta Hedging via Deep Learning. arXiv:2407.19367, version 2. doi:10.48550/arXiv.2407.19367
- [21] Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. 2020. Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., Red Hook, NY, USA, 7537–7547. <https://proceedings.neurips.cc/paper/2020/hash/55053683268957697aa39fba6f231c68-Abstract.html>
- [22] Kirill Zernikov. 2026. What Does Deep Hedging Actually Learn? Delta Corrections, Regime Fragility, and Symbolic Distillation. arXiv:2605.21696. doi:10.48550/arXiv.2605.21696